

Proje Yönetim ve Geliştirme Master Planı

PROJE YÖNETİM VE GELİŞTİRME MASTER PLANI

Bitki Hastalığı Tahmin Sistemi — Yapay Zeka ve Teknoloji Akademisi Bootcamp 2026

- PROJE ÖZETİ VE HEDEFLER
- ROLLER VE NET GÖREV DAĞILIMLARI
- TEKNİK MİMARİ VE ALTYAPI PLANI
- DETAYLI YOL HARİTASI (SPRINT PLANLAMASI)
- GÜNLÜK SCRUM AKIŞI VE RUTİNLER
- JÜRİ VE PUANLAMA KONTROL LİSTESİ
- EKLER: ŞABLONLAR

PROJE YÖNETİM VE GELİŞTİRME MASTER PLANI

Bitki Hastalığı Tahmin Sistemi — Yapay Zeka ve Teknoloji Akademisi Bootcamp 2026

Alan	Bilgi
Proje Kategorisi	Yapay Zeka & Veri Bilimi (10. Tema: Bitki Hastalığı Tahmin Sistemi)
Metodoloji	Scrum (3 Sprint x 2 Hafta)
Ekip Büyüklüğü	5 Kişi (1 PO, 1 SM, 3 Developer — herkes kod yazar)
Toplam Süre	6 Hafta (19 Haziran – 2 Ağustos 2026)
Doküman Versiyonu	v1.0
Doküman Amacı	Ekibin proje boyunca referans alacağı tek kaynak (single source of truth)

Nasıl kullanılır: Bu doküman; vizyon, roller, teknik mimari, sprint yol haritası, günlük rutinler ve jüri puanlama kontrol listesini tek çatı altında toplar. Sprint başlarında ilgili bölüm gözden geçirilmeli, sprint sonlarında “Jüri ve Puanlama Kontrol Listesi” bölümündeki maddeler tek tek işaretlenmelidir.

1. PROJE ÖZETİ VE HEDEFLER

1.1 Vizyon

Anlık durum tespiti yapan klasik bir sınıflandırma sisteminin ötesine geçerek, çiftçiye **“bitkiniz önümüzdeki 5 gün içinde %72 olasılıkla hastalanacak”** diyebilen, önleyici (predictive) bir karar destek sistemi geliştirmek. Sistem yalnızca teşhis koymaz; geçmiş verileri hafızasında tutan bir **Orkestratör Ajan** aracılığıyla kişiselleştirilmiş tavsiye üretir ve olası verim kaybını finansal olarak öngörür.

1.2 Çözümün Özeti

Bileşen	Görev
CNN Modeli	Yaprak görüntüsünden anlık hastalık/sağlık durumu tespiti
LSTM Modeli	Hava durumu, nem, sıcaklık gibi zaman serisi verilerinden 5 günlük risk tahmini
Ensemble Katmanı	CNN + LSTM çıktısını birleştirip nihai olasılık skorunu üretme
Regresyon Modülü	Hastalık riskine bağlı tahmini rekolte (verim) kaybını hesaplama
Orkestratör Ajan	Kullanıcı etkileşimini yönetme, ilgili modelleri tetikleme, hafızayı (geçmiş veri) kullanarak tavsiye üretme
Güvenlik Katmanı	Veri sınıflandırması ve rol bazlı yetkilendirme (RBAC)
Platform	Canlıya alınmış web/mobil arayüz

Not: Ürünün adı, tam kapsamı ve hedef kitlesi ekip tarafından netleştirilmelidir (bkz. Bölüm 7.1 Ürün Fikri Şablonu). Bu doküman boyunca proje “Sistem” olarak anılacaktır; ekip ilk 48 saat içinde resmi bir ürün adı belirlemelidir.

1.3 Jüriyi Etkileyecek Kilit Noktalar

Bootcamp kaynağındaki puanlama tablolarına göre, YZ kategorisinde **toplam puanın en büyük dilimi (Final değerlendirmede 35/100) doğrudan “Yapay Zeka Öğeleri” başlığına aittir.** Bu, ekibin zamanının büyük kısmını şu noktalara ayırması gerektiği anlamına gelir:

- Agent orkestrasyonu görünür ve gerçek olmalı** — süslemek için eklenmiş bir LLM çağrısı değil, CNN/LSTM tetikleme + hafıza kullanımı + tavsiye üretimini uçtan uca yöneten işlevsel bir ajan.
- Ensemble mimarisi net şekilde belgelenmeli** — CNN ve LSTM’in nasıl birleştiği (ör. ağırlıklı ortalama, stacking, geç füzyon) README’de ve koddan açık olmalı.
- Hafıza (memory) mekanizması** — çiftçinin geçmiş verilerini gerçekten saklayıp bir sonraki tavsiyede kullanmalı; dekoratif olmamalı.
- Temiz kod ve mimari yapı** — katmanlı mimari (data/model/agent/api/ui ayrımı), tutarlı isimlendirme, gereksiz tekrarın olmaması.
- Deployment** — ürün gerçekten canlıda çalışır durumda olmalı veya kolayca canlıya alınabilecek şekilde hazır olmalı.
- İhtiyaç-Çözüm eşleşmesi** — çiftçinin gerçek bir problemine (geç teşhis, finansal belirsizlik) doğrudan cevap verdiği anlatılmalı.

1.4 Resmi Puanlama Tabloları (Referans)

Ön Değerlendirme - YZ Kategorisi

Jüri Kriteri	Max Puan	Academy Club Ek Puan Kriteri	Max Puan
Yarışmaya hazır, çalışan proje	10	Yapay Zeka Modeli seçimi, kullanımı, geliştirmesi	20
Özgünlük	10	Agent kullanımı, hafıza, orkestrasyon yönetimi	15
Ürün tamamlanma puanı	10	Mimari yapı, temiz kod prensipleri	15
Pazara uygun, talep görebilecek uygulama	10	Ürün canlıya alınmış/alınabilecek durumda	10

Final Değerlendirme - YZ Kategorisi

Jüri Kriteri	Max Puan
İhtiyaç ve Çözüm Eşleşmesi	20
Kullanıcı Değeri ve Deneyimi	10
Pazar Potansiyeli	10

Fonksiyonel Yeterlilik	15
Ürün Bütünlüğü	10
Yapay Zeka Öğeleri	35

Kritik kural: Hazır proje kullanmak, satın alma yapmak veya dışarıdan kod desteği almak **diskalifiye sebebidir**. Tüm kod ekip tarafından sıfırdan yazılmalı ve GitHub'da adım adım belgelenmelidir.

1.5 Takvim Özeti

Aşama	Tarih
Takımların Açıklanması	12 Haziran 2026
Sprint 1	19 Haziran – 5 Temmuz 2026
Ara Soru-Cevap	6 Temmuz 2026, 20:00
Sprint 2	6 Temmuz – 19 Temmuz 2026
Ara Soru-Cevap	20 Temmuz 2026, 20:00
Sprint 3 (Son Sprint)	20 Temmuz – 2 Ağustos 2026
Son Teslim	2 Ağustos 2026, 23:59
Top 10 Sunum	14 Ağustos 2026

2. ROLLER VE NET GÖREV DAĞILIMLARI

Ekipte hiyerarşik bir lider yoktur. PO ve SM, süreç sorumluluklarının **yanında** aktif olarak geliştirmeye katılır. Kimin hangi kodu yazacağı ekibin ortak kararıdır; aşağıdaki dağılım bir **başlangıç önerisidir**, ekip Sprint 1'in ilk günü bunu netleştirmelidir.

2.1 Genel Sorumluluk Matrisi

Rol	Süreç Sorumluluğu	Önerilen Teknik Odak
Product Owner (PO)	Vizyon, backlog yönetimi, önceliklendirme, user story yazımı	Veri toplama & ön işleme, regresyon (verim kaybı) modülü
Scrum Master (SM)	İletişim, Scrum event yönetimi, blocker takibi, final Ürün Teslim Formu	Orkestratör Ajan mimarisi, memory entegrasyonu
Developer 1	Geliştirme	CNN modeli (görüntü sınıflandırma)
Developer 2	Geliştirme	LSTM modeli (zaman serisi) + ensemble birleştirme
Developer 3	Geliştirme	Frontend/Backend entegrasyonu, RBAC, deployment

2.2 Product Owner (PO) — Günlük & Haftalık Görevler

Günlük: - Backlog'daki story'lerin durumunu board üzerinde kontrol etmek. - Geliştirme sırasında ortaya çıkan kapsam sorularını (ör. "bu özelliği MVP'ye alalım mı?") aynı gün yanıtlamak. - Kendi geliştirme görevine (veri/regresyon modülü) en az 1-2 saat ayırmak.

Haftalık: - Backlog refinement: bir sonraki sprint için story'leri netleştirmek, önceliklendirmek. - Sprint Review'da tamamlanan işi "ihtiyaç-çözüm eşleşmesi" perspektifinden değerlendirmek (jüri gözüyle bakmak). - Hedef kitle ve pazar potansiyeli anlatısını (jüri kriteri) güncel tutmak; README'deki "Ürün Açıklaması" ve "Hedef Kitle" bölümlerinin güncel kalmasını sağlamak.

2.3 Scrum Master (SM) — Günlük & Haftalık Görevler

Günlük: - Daily Scrum'ı başlatmak/derlemek (Slack üzerinden asenkron da olabilir) ve notları GitHub'a/README'ye eklemek. - Blocker olup olmadığını sormak; varsa 24 saat içinde çözüm yolu bulmak veya ilgili kişilerle eşleştirmek. - Kendi geliştirme görevine (Agent orkestrasyonu) zaman ayırmak.

Haftalık: - Sprint Review ve Sprint Retrospective toplantılarını planlamak ve yönetmek. - Sprint Board'un (Miro/Trello) güncel olduğunu doğrulamak. - Gerekirse Academy Club eğitim asistanıyla ofis saatinde görüşüp süreçle ilgili soruları iletmek. - Sprint sonunda GitHub'a: Backlog dağıtma mantığı, Daily Scrum notları, Sprint Board ekran görüntüleri, ürün durumu görselleri, Review/Retrospective özetlerini yüklemek (bkz. Bölüm 6).

2.4 Developer'lar (3 Kişi) — Günlük & Haftalık Görevler

Günlük (herkes için ortak): - Üstlendiği story üzerinde ilerlemek; board'da kartın durumunu güncellemek (To Do → In Progress → Done). - Bir blocker varsa **aynı gün** SM'ye bildirmek. - Kod push etmeden önce kısa bir self-review yapmak (isimlendirme, gereksiz kod, yorum satırları).

Haftalık: - Sprint Planning'de kendi kapasitesine göre story tahmini yapmak. - Diğer developerlarla entegrasyon noktalarını (ör. CNN çıktısının ensemble'a nasıl bağlanacağı) senkronize etmek. - Sprint Review'da kendi geliştirdiği parçayı demo etmek.

Teknik iş bölümü önerisi:

Developer	Sprint 1 Odağı	Sprint 2 Odağı	Sprint 3 Odağı
Dev 1 (CNN)	Veri seti hazırlama, temel CNN prototipi	Model iyileştirme, ensemble'a bağlanacak API çıktısı	Test, model optimizasyonu, deployment desteği
Dev 2 (LSTM)	Zaman serisi veri şeması, temel LSTM prototipi	Ensemble birleştirme katmanı, regresyon entegrasyonu	Test, performans iyileştirme
Dev 3 (Full-stack)	UI/UX iskeleti, backend API taslağı	Agent ile backend entegrasyonu, RBAC iskeleti	Deployment, uçtan uca entegrasyon testleri

3. TEKNİK MİMARİ VE ALTYAPI PLANI

3.1 Veri Akışı (Yüksek Seviye)



3.2 Bileşen Detayları

Bileşen	Açıklama	Önerilen Teknoloji
Görüntü Modeli	Yaprak fotoğrafından hastalık sınıflandırması	PyTorch / TensorFlow-Keras, transfer learning (ResNet/EfficientNet tabanlı)
Zaman Serisi Modeli	Hava/nem/sıcaklık + geçmiş hastalık kayıtlarından 5 günlük risk tahmini	PyTorch/Keras LSTM, ya da basitleştirilmiş GRU
Ensemble Katmanı	İki modelin çıktısını birleştirme	Ağırlıklı ortalama veya küçük bir meta-model (stacking)
Regresyon Modülü	Risk skorundan tahmini verim kaybı (%) hesaplama	Scikit-learn (Linear/Gradient Boosting Regressor)
Orkestratör Ajan	Araç çağırma (tool-calling), hafıza yönetimi, tavsiye metni üretimi	Python tabanlı agent framework (ör. LangChain/LangGraph) + bir LLM API
Hafıza Deposu	Çiftçi bazlı geçmiş kayıtlar	PostgreSQL / SQLite (basit) veya vektör veritabanı (ör. Chroma) semantik geçmiş için
Backend API	Model ve agent servislerini dışa açma	FastAPI
Frontend	Kullanıcı arayüzü	Streamlit (hızlı prototipleme) veya React
Kimlik/RBAC	Rol bazlı erişim (çiftçi / danışman / admin)	JWT tabanlı auth + rol kontrolü middleware
Deployment	Canlı ortam	Streamlit Community Cloud, Render, veya AWS (EC2/Elastic Beanstalk)

LLM seçimi notu: Orkestratör ajanın doğal dil tavsiye üretimi için bir LLM API'sine ihtiyacı olacaktır. Öğrenci bütçesi ve hız/maliyet dengesi göz önüne alındığında ekip, düşük maliyetli bir “flash/mini” sınıfı model (ör. Gemini'nin hafif modeli) ile başlayıp gerekirse daha güçlü bir modele geçmeyi değerlendirebilir. Hangi sağlayıcının seçileceği ekibin ortak kararıdır; kritik olan seçimin README'de gerekçelendirilmesidir (bu, “Yapay Zeka Modeli seçimi” puanlama kriterine doğrudan katkı sağlar).

3.3 Agent Orkestrasyonu — Akış Mantığı

- Kullanıcı fotoğraf + (opsiyonel) konum/çiftçi kimliği yükler.
- Orkestratör Ajan, çiftçinin hafızasını (varsa geçmiş kayıtları) sorgular.
- Ajan, CNN modelini görüntü üzerinde, LSTM modelini güncel hava verisi üzerinde **paralel** tetikler (tool-calling).
- İki model çıktısı ensemble katmanında birleştirilir.
- Nihai risk skoru regresyon modülüne girdi olur; tahmini verim kaybı hesaplanır.
- Ajan, hafızadaki geçmiş verilerle (ör. “bu çiftçinin geçen ay da benzer bir vakası vardı”) bağlamsallaştırılmış bir tavsiye metni üretir.
- Sonuç, RBAC katmanından geçerek kullanıcının rolüne uygun detay seviyesinde arayüze döner.

8. Etkileşim ve sonuç, bir sonraki tavsiye için hafızaya yazılır.

3.4 Deployment Adımları (Taslak)

1. Model dosyalarını (CNN/LSTM/regresyon) versiyon kontrolüne uygun şekilde (ör. `models/` klasörü, Git LFS gerekirse) repoya eklemek.
2. Backend API'yi (FastAPI) konteynerize etmek (Dockerfile).
3. Frontend'i (Streamlit/React) backend'e bağlamak, ortam değişkenlerini (`.env`) yönetmek — **API anahtarları asla repoya commit edilmemeli.**
4. Seçilen platforma (Render/Streamlit Cloud/AWS) dağıtmak.
5. Canlı URL'yi README'ye ve final teslim formuna eklemek.
6. Temel yük/duman testi (smoke test) yaparak uçtan uca akışın çalıştığını doğrulamak.

3.5 Güvenlik ve RBAC Notu

- Roller örneğin: **Çiftçi** (yalnızca kendi verisini görür), **Danışman/Uzman** (birden fazla çiftçinin özet verisini görür), **Admin** (tüm sistem).
 - Veri sınıflandırması: kişisel veri (çiftçi bilgisi) ile model çıktısı (tahmin skorları) ayrı şemalarda tutulmalı.
 - Bu katman “amaç dışına çıkmadan” sade ama gerçek şekilde uygulanmalı; sahte/dekoratif bir rol kontrolü puan kaybettirir.
-

4. DETAYLI YOL HARİTASI (SPRINT PLANLAMASI)

İş yükü, 5 kişinin bootcamp dışında eğitim/iş sorumlulukları olabileceği varsayımıyla **gerçekçi** şekilde dağıtılmıştır. Her sprint sonunda Board, GitHub ve README güncellenmelidir.

4.1 Sprint 1 — Temel Atma (19 Haziran - 5 Temmuz, ~2.5 hafta)

Sprint Hedefi: Mimarinin iskeletini kurmak; her iki model için çalışan (henüz optimize edilmemiş) birer prototip üretmek; proje yönetim altyapısını (GitHub, Board, README) tam işler hale getirmek.

Epic 1 — Proje Kurulumu ve Yönetişim - Story: GitHub reposu oluşturma, README iskeleti, branch stratejisi belirleme (ör. `main` + feature branch). - Story: Sprint Board (Miro/Trello) kurulumu ve backlog'un ilk sürümünün board'a taşınması. - Story: Ürün fikrinin, takım rollerinin ve hedef kitlenin README'de belgelenmesi (bkz. Ek 7.1).

Epic 2 — Veri Altyapısı - Story: Yaprak görüntü veri setinin belirlenmesi/toplanması ve ön işlenmesi. - Story: Hava durumu / nem / sıcaklık verisi için kaynak belirleme (API veya açık veri seti) ve şema tasarımı. - Story: Geçmiş hastalık kayıtları için basit bir veri şeması oluşturma (agent hafızasının temeli).

Epic 3 — Model Prototipleri - Story: CNN modeli için ilk (baseline) eğitim denemesi. - Story: LSTM modeli için ilk (baseline) eğitim denemesi.

Epic 4 — Ürün İskeleti - Story: Backend API iskeleti (FastAPI) — boş endpoint'ler. - Story: Frontend iskeleti (Streamlit/React) — fotoğraf yükleme ekranı taslağı.

Definition of Done (Sprint 1): Repo public, README'nin "Takım İsmi / Ürün İsmi / Ürün Açıklaması / Hedef Kitle" bölümleri dolu, board'da tüm epic'ler story'lere bölünmüş, iki model için de en az bir "çalışıyor" (accuracy düşük olsa da) prototip var.

4.2 Sprint 2 — Entegrasyon ve Zeka (6 Temmuz - 19 Temmuz, 2 hafta)

Sprint Hedefi: CNN + LSTM ensemble'ını birleştirmek; Orkestratör Ajanı gerçek tool-calling ve hafıza ile çalışır hale getirmek; regresyon modülünü entegre etmek.

Epic 5 — Ensemble ve Regresyon - Story: CNN ve LSTM çıktılarını birleştiren ensemble katmanının yazılması. - Story: Verim kaybı regresyon modelinin eğitilmesi ve ensemble skoruna bağlanması.

Epic 6 — Agent Orkestrasyonu - Story: Orkestratör ajanının CNN/LSTM/regresyon modüllerini tool olarak çağırabilmesi. - Story: Hafıza mekanizmasının (çiftçi geçmişi) agent'a bağlanması. - Story: Agent'ın bağlamsallaştırılmış tavsiye metni üretmesi.

Epic 7 — Backend/Frontend Entegrasyonu - Story: Backend API'nin agent ile uçtan uca bağlanması. - Story: Frontend'de sonuç ekranının (risk skoru, verim kaybı tahmini, tavsiye metni) tasarlanması.

Epic 8 — Güvenlik Temeli - Story: RBAC iskeletinin (rol tanımları, yetkilendirme middleware) yazılması.

Definition of Done (Sprint 2): Fotoğraf + hava verisi girişiyle uçtan uca bir tahmin ve tavsiye üretilebiliyor; agent gerçekten iki modeli de tetikliyor; hafıza en az bir senaryoda tavsiyeyi etkiliyor.

4.3 Sprint 3 — Cilalama ve Teslim (20 Temmuz - 2 Ağustos, 2 hafta)

Sprint Hedefi: Ürünü canlıya almak, temiz kod/mimari son halini vermek, tüm dokümantasyonu ve teslim materyallerini tamamlamak.

Epic 9 — Kalite ve Optimizasyon - Story: Model performans iyileştirmeleri (mümkünse). - Story: Kod tabanının refactor edilmesi (temiz kod prensipleri, katman ayrımı). - Story: Temel test senaryolarının yazılması (en azından kritik akış için).

Epic 10 — Deployment - Story: Uygulamanın seçilen platforma deploy edilmesi. - Story: Canlı ortamda uçtan uca smoke test.

Epic 11 — Final Teslim - Story: README'nin tüm bölümlerinin (mimari, kurulum, kullanım) tamamlanması. - Story: 3 dakikalık proje tanıtım videosunun çekilip YouTube'a yüklenmesi. - Story: Final teslim formunun (Ürün Teslim Formu — SM sorumluluğunda) doldurulması. - Story: Sprint 3 Review & Retrospective belgelerinin GitHub'a eklenmesi.

Definition of Done (Sprint 3): Ürün canlıda çalışıyor (veya canlıya alınabilir durumda), GitHub kayıtları eksiksiz, video yüklendi, form 2 Ağustos 23:59'dan önce gönderildi.

4.4 GitHub ve Board Kullanım Adımları (Her Sprint İçin Ortak)

- Sprint başında backlog'daki story'ler board'da "To Do" sütununa taşınır.
- Bir developer story'yi üstlendiğinde kartı kendine atar ve "In Progress"e taşır.

3. Her commit, ilgili story/issue numarasına referans verir (ör. `feat: CNN baseline model (#12)`).
4. Story tamamlandığında PR açılır, en az bir takım arkadaşı review yapar, `main` 'e merge edilir.
5. Sprint sonunda: Board'un ekran görüntüsü + Backlog dağıtma mantığı açıklaması README'ye/`docs/sprintX.md` dosyasına eklenir.

4.5 Sprint Review / Retrospective Süreci

Sprint Review (yaklaşık 30 dk): - Tamamlanan story'lerin canlı/ekran görüntüsü ile demo edilmesi. - PO'nun "ihtiyaç-çözüm eşleşmesi" açısından geri bildirimi. - Bir sonraki sprint için açık noktaların not edilmesi.

Sprint Retrospective (yaklaşık 20 dk): - "İyi gitti / zorlandık / deneyeceğiz" formatında 3 sütunlu değerlendirme. - En az 1 somut aksiyon maddesi belirlenip bir sonraki sprint'e taşınması.

5. GÜNLÜK SCRUM AKIŞI VE RUTİNLER

5.1 Daily Scrum Kuralları

- Format:** Ekip yoğun ders/iş programına sahip olabileceğinden, Daily Scrum **eşzamansız (asenkron)** olarak Slack üzerinden yürütülebilir; haftada en az 1-2 kez canlı (video/sesli) görüşme önerilir.
- Zaman:** Her gün belirlenen sabit bir saatte (ör. akşam 21:00) mesaj atılır.
- İçerik (3 Soru):**
 - Dün ne yaptım?
 - Bugün ne yapacağım?
 - Önümde bir engel (blocker) var mı?
- Sorumlu:** SM, notları derleyip haftalık olarak GitHub'a (docs/daily-scrum/) ekler.

5.2 İletişim Kanalları

Kanal	Amaç
Takım Slack/WhatsApp grubu	Günlük iletişim, Daily Scrum
GitHub Issues/PR	Teknik tartışma, kod review
Sprint Board (Miro/Trello)	İş takibi, görsel durum
Haftalık video görüşme	Sprint Planning, Review, Retrospective
#bootcamp-2026 (Slack)	Akademi/Academy Club ile iletişim (proje yönetimi soruları, 48 saat içinde yanıt)

5.3 Blocker Yönetimi

- Blocker fark edildiği an takım kanalına yazılır (beklenmez).
- SM, 24 saat içinde ya çözüm önerir ya da ilgili kişiyle eşleştirir.
- 48 saatten uzun süredir çözilemeyen teknik blocker'lar için Academy Club eğitim asistanının ofis saatinde soru sorulur (asistanlar teknik konularda özel destek vermez, ancak proje/GitHub/Agile yönlendirmesi yapabilir).

6. JÜRİ VE PUANLAMA KONTROL LİSTESİ

6.1 Her Sprint Sonunda Kontrol Edilecekler (Proje Yönetimi Puanı İçin)

- ☐ Backlog dağıtma mantığı GitHub'da açıklandı mı? (Hangi story'ler neden seçildi, tahmin puanları neye göre verildi?)
- ☐ Daily Scrum notları eklendi mi?
- ☐ Sprint Board güncel ve ekran görüntüsü GitHub'a eklendi mi?
- ☐ Ürün durumu (ekran görüntüleri/kısa video) paylaşıldı mı?
- ☐ Sprint Review özeti yazıldı mı?
- ☐ Sprint Retrospective özeti ve aksiyon maddesi yazıldı mı?
- ☐ README güncellendi mi?

Not: Bootcamp'te proje yönetimi puanı ile ürün puanı **ayrı ayrı** değerlendirilir; 3 sprint sonundaki toplam bu kontrol listesinden doğar.

6.2 Final Teslim Kontrol Listesi

- ☐ GitHub reposu public ve tüm sprint kayıtları eksiksiz.
- ☐ README: Takım ismi, roller, ürün ismi, açıklama, özellikler, hedef kitle, backlog linki dahil tüm bölümler dolu.
- ☐ 3 dakikalık proje videosu YouTube'a yüklendi.
- ☐ Ürün canlıya alındı (veya canlıya alınabilir durumda, link paylaşıldı).
- ☐ Final teslim formu eksiksiz dolduruldu (Scrum Master sorumluluğunda).
- ☐ Tüm teslimler **2 Ağustos 2026, 23:59**'dan önce tamamlandı.

6.3 Yapay Zeka Öğeleri Kontrol Listesi (35 Puanlık Kritik Alan)

- ☐ CNN modeli gerçekten eğitildi ve çalışıyor mu?
- ☐ LSTM modeli gerçekten eğitildi ve çalışıyor mu?
- ☐ Ensemble birleştirme mantığı kodda ve README'de net mi?
- ☐ Orkestratör Ajan, modelleri gerçek tool-calling ile tetikliyor mu (sahte/dekoratif değil)?
- ☐ Hafıza (memory) mekanizması en az bir senaryoda tavsiyeyi somut şekilde etkiliyor mu?
- ☐ Regresyon modülü (verim kaybı) ensemble skoruna bağlı çalışıyor mu?
- ☐ Kod mimarisi katmanlı ve temiz mi (model/agent/api/ui ayrımı net mi)?

6.4 Diskalifiye Riski Kontrolü

- ☐ Hazır/şablon bir proje kopyalanmadı.
- ☐ Herhangi bir bileşen satın alınmadı.
- ☐ Dışarıdan kod desteği alınmadı.
- ☐ Tüm geliştirme adımları GitHub commit geçmişiyle kanıtlanabilir.

7. EKLER: ŞABLONLAR

7.1 README — Ürün Fikri ve Roller Şablonu

```
# [Takım İsmi]

## Ürün İle İlgili Bilgiler

### Takım Elemanları
- [Ad Soyad]: Product Owner
- [Ad Soyad]: Scrum Master
- [Ad Soyad]: Developer
- [Ad Soyad]: Developer
- [Ad Soyad]: Developer

### Ürün İsmi
--[Ürün Adı]--

### Ürün Açıklaması
[Ürünün ne yaptığını, hangi problemi çözdüğünü 2-3 cümlede açıklayın.]

### Ürün Özellikleri
- [Özellik 1]
- [Özellik 2]
- [Özellik 3]

### Hedef Kitle
- [Kitle 1]
- [Kitle 2]

### Product Backlog URL
[Miro/Trello Board Linki]
```

7.2 Daily Scrum Notu Şablonu

```
## Daily Scrum – [Tarih]

| Kişi | Dün Yaptığım | Bugün Yapacağım | Blocker |
|---|---|---|---|
| [Ad] | ... | ... | Yok/Var: ... |
```

7.3 Sprint Review/Retrospective Şablonu

```
## Sprint [N] Review

**Tamamlanan Story'ler:**
- ...

**Demo Notları:**
- ...

## Sprint [N] Retrospective

| İyi Gitti | Zorlandık | Deneyeceğiz |
|---|---|---|
| ... | ... | ... |

**Aksiyon Maddesi:** ...
```

7.4 Örnek Repo Referansları (Bootcamp Kaynağından)

- <https://github.com/YapayZekaveTeknolojiAkademisi/BootcampScrumTemplate/tree/main>
- <https://github.com/kevsoOther/U-21-Cherry-Chasers/tree/main>
- <https://github.com/olgnbrn/planova>
- <https://github.com/lbrmSerhat/GhostOfAnnaScrumExample>
- <https://github.com/burakcevheroglu/OUA-zaten-Bootcamp-2023>

